

Problem 1

Consider a network of $N \in \mathbb{N}$ agents and the interaction between them is modelled via a graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$. Let the corresponding row-stochastic adjacency matrix be denoted by $A \in \mathbb{R}^{N \times N}$. Let $v \in \mathbb{R}_{\geq 0}^N$ be the normalised left dominant eigenvector such that $v^\top A = v^\top$ and $\mathbf{1}_N^\top v = 1$. For the consensus system $x[k+1] = Ax[k]$, the disagreement vector $\delta[k] \in \mathbb{R}^N$ at time $k \in \mathbb{N}$ is

$$\delta[k] = x[k] - (v^\top x[k])\mathbf{1}_N$$

Note that $v^\top x[k]$ corresponds to the normalized weighted average of the entries of $x[k]$ at time k .

1. Prove that $v^\top x[k]$ is constant over all time steps $k \in \mathbb{N}$.
2. Show that $(A - \mathbf{1}_N v^\top) = (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)$
3. Show that the dynamics of the disagreement vector is given by

$$\delta[k+1] = (A - \mathbf{1}_N v^\top)\delta[k]$$

Solution:

1. Since $x[k+1] = Ax[k]$ and $v^\top A = v^\top$,

$$\begin{aligned} v^\top x[k+1] &= v^\top Ax[k] \\ &= v^\top x[k] \end{aligned}$$

Hence, $v^\top x[k]$ is constant for all $k \in \mathbb{N}$.

2. Since A is row-stochastic, $A\mathbf{1}_N = \mathbf{1}_N$. Also, $v^\top \mathbf{1}_N = 1$ (Given). Hence,

$$\begin{aligned} (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top) &= A - \mathbf{1}_N v^\top - A\mathbf{1}_N v^\top + \mathbf{1}_N v^\top \mathbf{1}_N v^\top \\ &= A - \mathbf{1}_N v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N v^\top \\ &= A - \mathbf{1}_N v^\top \end{aligned}$$

Therefore,

$$A - \mathbf{1}_N v^\top = (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)$$

3. From the definition of $\delta[k]$,

$$\begin{aligned} \delta[k+1] &= x[k+1] - (v^\top x[k+1])\mathbf{1}_N \\ &= Ax[k] - (v^\top Ax[k])\mathbf{1}_N \\ &= (A - \mathbf{1}_N v^\top)x[k] \end{aligned}$$

And we also have, $x[k] = \delta[k] + (v^\top x[k])\mathbf{1}_N$ Therefore,

$$\begin{aligned} \delta[k+1] &= (A - \mathbf{1}_N v^\top) \left(\delta[k] + (v^\top x[k])\mathbf{1}_N \right) \\ &= (A - \mathbf{1}_N v^\top)\delta[k] + (v^\top x[k])(A - \mathbf{1}_N v^\top)\mathbf{1}_N \end{aligned}$$

Since

$$\begin{aligned}
(A - \mathbf{1}_N v^\top) \mathbf{1}_N &= A \mathbf{1}_N - \mathbf{1}_N (v^\top \mathbf{1}_N) \\
&= \mathbf{1}_N - \mathbf{1}_N \\
&= 0
\end{aligned}$$

we obtain $\delta[k+1] = (A - \mathbf{1}_N v^\top) \delta[k]$

Problem 2

Simulate the following tasks and give your inferences and all the associated codes.

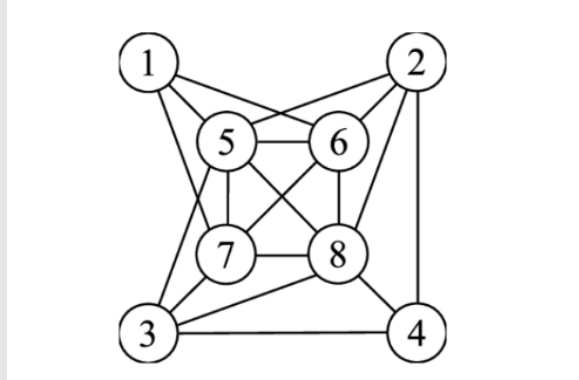


Figure 1:

1. Consider the above 3-robust graph. This offers robustness against $F = 1$ local adversaries in the network. You can assume local adversaries such that each node is allowed to have at most one bad friend. Assume random initial conditions and the adversary is free to do anything. Simulate the resilient consensus of this graph topology using W-MSR protocol.
2. Argue by constructing/drawing subsets to show why this 3-robust graph is *not* a $(3, 2)$ -robust graph. *Hint:* You might need to choose just that one partition S_1, S_2 to prove.
3. Suppose that there is an eavesdropper (aka Spy), who is listening and has access to all the communication links. Note that the eavesdropper is an external entity of the network - who can be either good or bad. For instance, cyber-police are technically “good” eavesdroppers as they can access our phones for national security purposes. Consider the case of Good eavesdropper who observes the communication, figures out who the malicious agent is and intervenes by secretly modifying the malicious agent’s communicated values to help the benign neighbors. How can then the malicious agent execute his attack? What could be the potential strategy of the malicious agent to get past the eavesdropper?
4. Consider the setting where benign agents somehow sense that there is some potential uncontrollable adversary in the network. So, they decide to implement a secure communication, where every node pair (i, j) in the network secretly implements a corresponding encrypted communication. That is, the node pair (i, j) have their own corresponding encryption/decryption setting. This is when you need the combination of both a resilient and a privacy-preserving consensus algorithm for multi-agent systems. Recall that in W-MSR, you do filtering by sorting the neighboring values and then discard the extreme F values and average the remaining ones. Should the benign agents perform W-MSR on the received encrypted neighbor values or should it decrypt the neighbor values first and then perform W-MSR. Justify which one of the above two options is a promising one than the other. What subsequent problems one could face with the promising strategy? There are already papers in this area. Find out and read 2 seminal papers

from the research literature to solve this problem and report your understanding (crisply in just 1 paragraph for each paper) of how the authors of the papers addressed this problem. You do not need to code anything here (unless you have all the time on this earth for you!). Remember, this is not a Cryptography class - so no need to report on what cryptographic algorithm was used in those papers. You can happily do that banter in Cryptography courses offered by the CS department.

Solution:

1. The given 3-robust graph is simulated using the W-MSR protocol with $F = 1$. Random initial conditions are assigned to all nodes, with Node 8 acting as the local adversary. The adversary does not follow the W-MSR protocol and varies its state according to a bounded oscillatory behavior with added random noise.

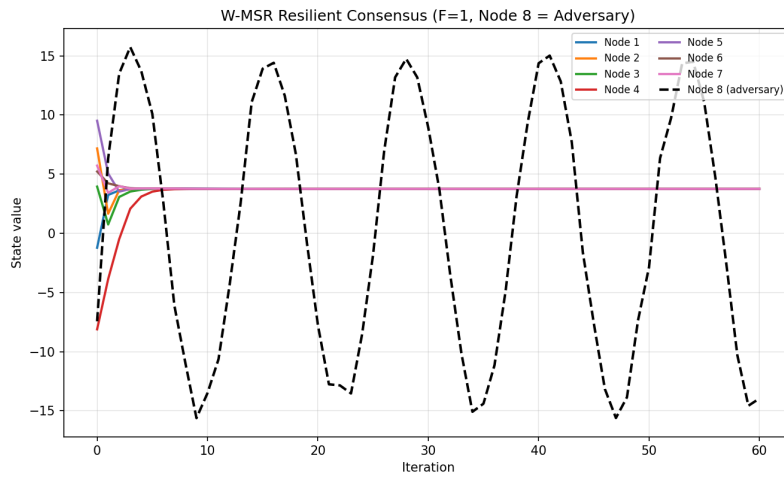


Figure 2: W-MSR resilient consensus with one local adversary.

From the simulation, the adversarial node (Node 8) continues to vary its state, while the states of the normal nodes (Nodes 1–7) converge to a common value. For the chosen random initial conditions, the final states at $t = 60$ are approximately

$$x_i \approx 3.7548, \quad i = 1, \dots, 7.$$

The spread among the normal nodes at the final iteration is

$$\max_i x_i - \min_i x_i = 0.$$

Thus, the normal nodes reach consensus despite the malicious behavior of Node 8. This demonstrates the resilience of the given 3-robust graph under the W-MSR protocol for $F = 1$.

The code used for the simulation in Problem 1 is given below:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Graph definition
5
6 neighbors = {
7     1: [5, 7, 6],

```

```

8     2: [5, 6, 8, 4],
9     3: [5, 7, 8, 4],
10    4: [2, 3, 8],
11    5: [1, 2, 3, 6, 7, 8],
12    6: [1, 2, 5, 7, 8],
13    7: [1, 3, 5, 6, 8],
14    8: [2, 3, 4, 5, 6, 7],
15 }
16
17 N = 8
18 F = 1
19 ADVERSARY = 8                # node acting as the adversary
20 NORMAL_NODES = [i for i in range(1, N + 1) if i != ADVERSARY]
21
22 T = 60                        # number of iterations
23 rng = np.random.default_rng(seed=42)
24
25 # Random initial conditions
26 x = rng.uniform(-10, 10, size=N + 1) # index 0 unused, nodes are 1..8
27 history = np.zeros((T + 1, N + 1))
28 history[0] = x.copy()
29
30 print("Initial values:")
31 for i in range(1, N + 1):
32     print(f"  Node {i}: {x[i]:.4f}")
33
34 # W-MSR update rule for one normal node
35 def wmsr_update(node, x_curr, F):
36     own_val = x_curr[node]
37     nbr_vals = [x_curr[j] for j in neighbors[node]]
38
39     # values strictly larger / smaller than own value
40     larger = sorted([v for v in nbr_vals if v > own_val], reverse=True)
41     smaller = sorted([v for v in nbr_vals if v < own_val])
42     middle = [v for v in nbr_vals if v == own_val] # ties, kept as-is
43
44     # remove up to F largest and up to F smallest
45     larger = larger[F:] if len(larger) > F else []
46     smaller = smaller[F:] if len(smaller) > F else []
47
48     kept = [own_val] + larger + smaller + middle
49     return float(np.mean(kept))
50
51
52 # Adversary behavior (node 8)
53 def adversary_update(t):
54     return 15 * np.sin(0.5 * t) + rng.normal(0, 1.0)
55
56 # Simulation loop
57
58 for t in range(1, T + 1):
59     x_new = x.copy()
60
61     for node in NORMAL_NODES:
62         x_new[node] = wmsr_update(node, x, F)
63
64     x_new[ADVERSARY] = adversary_update(t)
65
66     x = x_new
67     history[t] = x.copy()

```

```

68
69 # Results / inference
70
71 final_normal_vals = [history[T, i] for i in NORMAL_NODES]
72 spread = max(final_normal_vals) - min(final_normal_vals)
73
74 print("\nFinal values (t = {}):".format(T))
75 for i in range(1, N + 1):
76     tag = " (ADVERSARY)" if i == ADVERSARY else ""
77     print(f" Node {i}: {history[T, i]:.4f}{tag}")
78
79 print(f"\nSpread among normal nodes at final step: {spread:.6f}")
80 if spread < 1e-3:
81     print("=> Normal nodes reached resilient consensus, despite node 8's
82           malicious behavior.")
83 else:
84     print("=> Normal nodes have not fully converged yet (try increasing T).")
85
86 plt.figure(figsize=(9, 5.5))
87 for i in range(1, N + 1):
88     if i == ADVERSARY:
89         plt.plot(history[:, i], 'k--', linewidth=2, label=f"Node {i} (
90               adversary)")
91     else:
92         plt.plot(history[:, i], linewidth=1.8, label=f"Node {i}")
93
94 plt.xlabel("Iteration")
95 plt.ylabel("State value")
96 plt.title("W-MSR Resilient Consensus (F=1, Node 8 = Adversary)")
97 plt.legend(loc="upper right", ncol=2, fontsize=8)
98 plt.grid(alpha=0.3)
99 plt.tight_layout()
100 plt.savefig("wmsr_consensus.png", dpi=150)
101 print("\nPlot saved to wmsr_consensus.png")

```

2. To show that the graph is not $(3, 2)$ -robust, it is enough to construct one partition into two nonempty, disjoint sets for which there are fewer than two nodes having at least 3 neighbors outside their respective sets.

Consider

$$S_1 = \{1, 2, 5, 6, 7\} \quad S_2 = \{3, 4, 8\}$$

For S_1 , we count the neighbors of each node that lie outside S_1 , i.e., in S_2 :

$$|N_1 \cap S_2| = 0$$

$$|N_2 \cap S_2| = 2$$

$$|N_5 \cap S_2| = 2$$

$$|N_6 \cap S_2| = 1$$

$$|N_7 \cap S_2| = 2$$

Thus, no node in S_1 has at least 3 neighbors outside S_1 .

For S_2 , we count the neighbors outside S_2 , i.e., in S_1 :

$$|N_3 \cap S_1| = 2$$

$$|N_4 \cap S_1| = 1$$

$$|N_8 \cap S_1| = 4$$

Therefore, only node 8 has at least 3 neighbors outside S_2 .

Hence, for this partition, there is only one node in $S_1 \cup S_2$ with at least 3 neighbors outside its respective set. Since $(3, 2)$ -robustness requires at least two such nodes, the condition is violated.

Therefore,

The given graph is not $(3, 2)$ -robust

3. The malicious agent may attempt to bypass the good eavesdropper through the transmission of carefully selected values that will look like legitimate nodes' actions. The malicious agent may avoid transmitting obvious or highly malicious values and instead gradually affect the consensus process while being hard to distinguish from a legitimate node.

Another possible strategy involves the use of encrypted and authenticated communication. The encryption ensures that the eavesdropper is not able to see the real values being transmitted, while the authentication prevents it from changing or forging the messages.

4. The more promising strategy is to avoid decrypting the neighbor states before filtering, since this would reveal their private values. However, ordinary W-MSR cannot be directly performed on ciphertexts because sorting and comparing encrypted values is not straightforward. Thus, the key challenge is to modify the filtering step so that it can identify and discard malicious information without revealing the actual neighbor states. This also introduces additional computational overhead and may affect the accuracy and convergence of the consensus.

Paper 1: Secure and Privacy-Preserving Consensus. Ruan, Gao, and Wang use homomorphic encryption to perform the required pairwise consensus computations without revealing the agents' states. Their protocol allows encrypted state differences to be processed while preserving privacy, and guarantees consensus without requiring a trusted third party. Thus, the paper establishes that consensus updates can be performed securely without first exposing the states through decryption, although it does not directly solve the sorting-based filtering required by W-MSR. [1]

Paper 2: Resilient and Privacy-Preserving Consensus for Multi-Agent Systems. Huang, Wu, and Huang directly combine homomorphic encryption with the MSR principle through their HE-MSR algorithm. They note that conventional MSR requires access to the exact neighbor states, which violates privacy. Their solution uses encrypted communication and a half-weight mechanism, and filters messages using the received weighted state differences rather than the exposed neighbor states. This allows suspicious messages to be discarded while preserving both privacy and resilience against Byzantine attacks. [2]

References

- [1] M. Ruan, H. Gao, and Y. Wang, "Secure and Privacy-Preserving Consensus," *IEEE Transactions on Automatic Control*, vol. 64, no. 10, pp. 4035–4049, 2019. [link](#)
- [2] M. Huang, Y. Wu, and Q. Huang, "Resilient and Privacy-Preserving Consensus for Multi-Agent Systems," *Information Sciences*, vol. 700, pp. 121843, 2025. [link](#)

AI Usage Disclosure

In making this assignment, AI tools were used for the sake of making the matter clear and understandable, such as organizing and refining the text properly, improving LaTeX formatting, and making the math easier to follow.